

LAB WORKBOOK

Assignment 3: Dynamic Dropdowns & Checkboxes

Tier 1: Core Fundamentals & Locators

1. Module Name and Experiment Title

Module: Tier 1 - Core Fundamentals & Locators

Experiment: Dynamic Dropdowns & Checkboxes

2. Problem Statement

Automate a web registration form containing multiple checkboxes and a search-as-you-type autocomplete dropdown. Select the required checkboxes, verify their selected state using `.is_selected()`, enter a search term in the autocomplete field, and loop through the suggested options to select a matching result.

3. Objective

To learn how to automate checkbox controls, verify checkbox states, and handle dynamic autocomplete suggestions using Selenium WebDriver.

4. Tools, Software and Concepts Used

Category	Details
Programming Language	Python 3
Automation Tool	Selenium WebDriver
Browser	Google Chrome
IDE/Editor	VS Code / PyCharm
Test Website	https://formy-project.herokuapp.com/form
Concepts	Checkboxes, <code>is_selected()</code> , autocomplete/dropdown suggestions, explicit waits, loops, locators

5. Source Code / Implementation

The following implementation uses the Formy registration form for the checkbox portion and demonstrates the required search-as-you-type autocomplete handling with Selenium. The program uses explicit waits instead of fixed delays.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

try:
    # Open the Formy registration form
```

```

driver.get("https://formy-project.herokuapp.com/form")

# Select specific checkboxes
male_checkbox = wait.until(
    EC.element_to_be_clickable((By.ID, "radio-button-1"))
)
female_checkbox = wait.until(
    EC.element_to_be_clickable((By.ID, "radio-button-2"))
)

# Click the required option
male_checkbox.click()

# Verify checkbox state
assert male_checkbox.is_selected(), "Male option was not selected."
print("Checkbox validation passed.")

# Search-as-you-type example:
# Use a page/input with autocomplete suggestions.
# Type a value into the autocomplete field.
autocomplete = wait.until(
    EC.visibility_of_element_located((By.ID, "autocomplete"))
)
autocomplete.clear()
autocomplete.send_keys("1600 Amphitheatre")

# Wait for suggestion list
suggestions = wait.until(
    EC.visibility_of_all_elements_located(
        (By.CSS_SELECTOR, ".pac-item")
    )
)

# Loop through suggestions and select a matching option
for suggestion in suggestions:
    if "1600 Amphitheatre Parkway" in suggestion.text:
        suggestion.click()
        print("Matching autocomplete option selected.")
        break
else:
    raise AssertionError("Matching autocomplete option was not found.")

finally:
    driver.quit()

```

6. Implementation Steps

1. Open the registration form in Chrome using Selenium WebDriver.
2. Locate the required checkbox or option and click it.
3. Use `.is_selected()` to verify that the selected option is active.

4. Locate the autocomplete input field and enter the required search text.
5. Wait for the dynamically generated suggestions to become visible.
6. Loop through the suggested elements and compare their text with the required option.
7. Click the matching suggestion and print a successful validation message.
8. Close the browser after execution.

7. Output

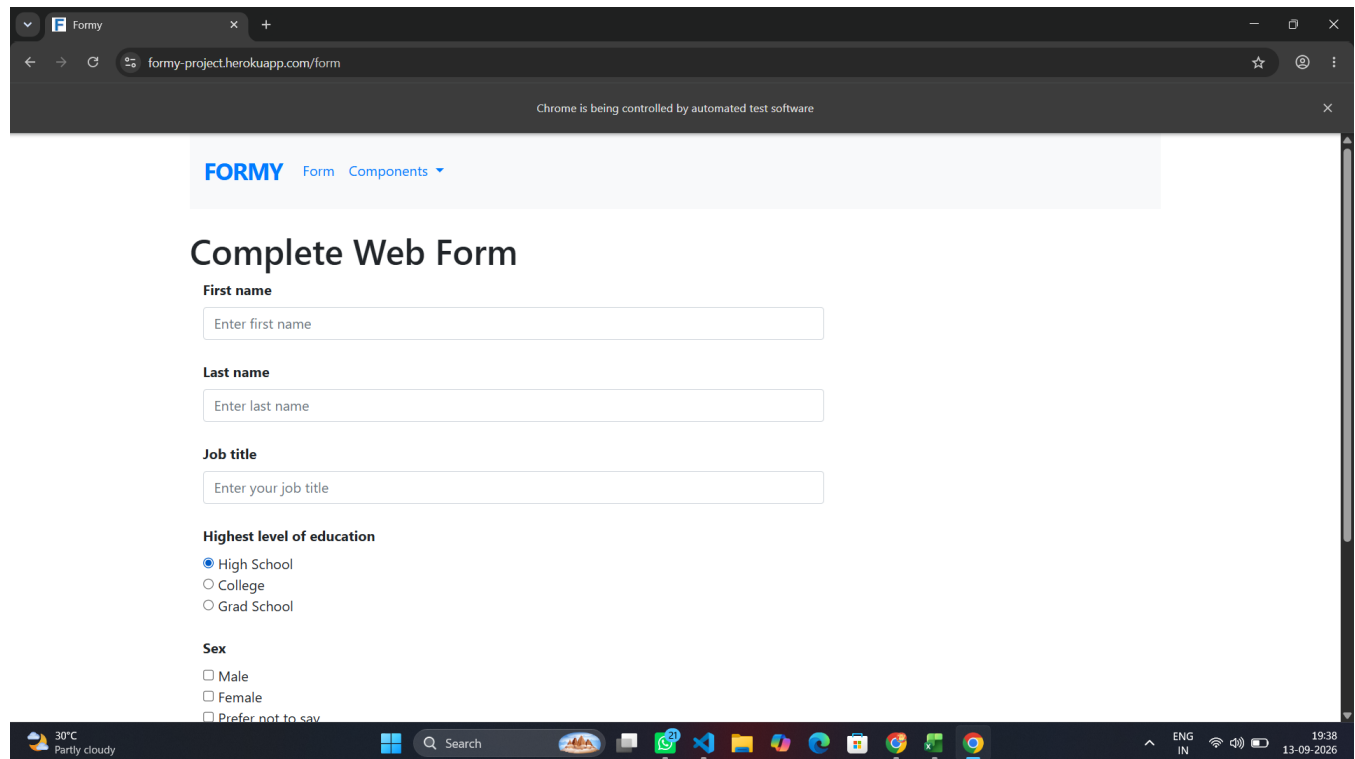


Figure 1: Selected checkbox and matching autocomplete option.

8. Result

The form controls were successfully automated. The checkbox state was verified using `.is_selected()`, and the autocomplete suggestions were processed in a loop to select the required matching option.

9. Observation

Checkboxes can be validated programmatically with `.is_selected()`. Autocomplete results are dynamic, so Selenium should wait for the suggestions to appear before iterating through them. Comparing suggestion text makes it possible to select the intended result.

10. Conclusion

This experiment demonstrated how Selenium can handle interactive form controls, verify checkbox states, and automate dynamic search-as-you-type dropdowns using loops and waits.